

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/353216919>

Image Classification with Convolutional Neural Networks

Chapter · July 2021

DOI: 10.1007/978-3-030-70542-8_18

CITATIONS

6

READS

1,140

4 authors:



Alfonso Ramos Michel

University of Guadalajara

16 PUBLICATIONS 65 CITATIONS

SEE PROFILE



Marco A. Pérez Cisneros

University of Guadalajara

201 PUBLICATIONS 4,037 CITATIONS

SEE PROFILE



Erik Cuevas

University of Guadalajara

387 PUBLICATIONS 7,769 CITATIONS

SEE PROFILE



Daniel Zaldivar

University of Guadalajara

112 PUBLICATIONS 3,206 CITATIONS

SEE PROFILE

Image classification with Convolutional Neural Networks

Alfonso Ramos-Michel^a, Marco Pérez-Cisneros^b, Erik Cuevas^c, Daniel Zaldivar^d

Division de Electronica y Computacion, Universidad de Guadalajara, CUCEI, Av. Revolución 1500, Guadalajara, Jalisco, Mexico, C.P. 44100;
alfonsorm@cucsur.udg.mx^a, marco.perez@cucei.udg.mx^b,
erik.cuevas@cucei.udg.mx^c, daniel.zaldivar@cucei.udg.mx^d

1 Abstract

The world, as we know, it would not exist without the development of machine learning (ML). Its application in processing information gives the human more advantages than in any other age. The use of ML brings the opportunity to perform categorizations into a big amount of elements and into areas that the human simply can not do it without this technology. But the development of ML algorithms is complex since it requires specific theoretical bases and the development of skills to achieve the tuning of these algorithms. Given the wide variety of algorithms and techniques to perform element classification, this chapter focuses on the study of Convolutional Neural Networks (CNN), and the development of tests that facilitate the understanding of the effects of their basic parameters. The classification objects used are images, and only the multiple class classification model is considered.

Keywords

Convolutional Neural Networks, Classification, Machine Learning

2 Introduction

In Artificial Intelligence (AI), you can distinguish a segment that tries to emulate the acquisition of knowledge of the animal brain. The area is known as ML and makes use of a series of algorithms capable of using the input information to adjust its behavior. The performance of these learning algorithms depends on a series of adjusted parameters before the processing of the information. For the computing scientist, it is essential to know the parameters that influence the learning process of a particular algorithm [7].

The knowledge acquired by the animal brain has a utility. Expands and improves the tasks it performs. For the human brain, the possibilities of using knowledge are practically endless. In the case of ML, it is common to use it as a support in decision-making or as a tool that favors the prediction of future situations [10]. In general, there are three main tasks to develop within the ML: Classification, clustering, and regression. A quick guide to finding out the type of task an ML algorithm performs is presented in the flowchart in Figure 1.

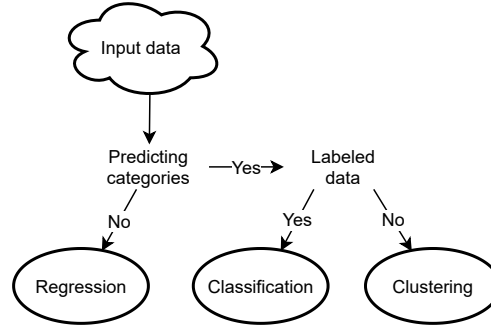


Fig. 1. Flowchart to identify the ML task that an algorithm performs.

The first division line of ML is categorization. If the algorithm does not perform a data categorization, then it performs a regression. If the algorithm categorizes the input information, then we can achieve a classification or clustering task. Both tasks make a grouping of elements. Clustering groups information without prior relational information through unsupervised learning. The classification uses supervised learning through labels.

The regression has the goal of generating a function based on history. Takes the known values of independent variables as input, and establish their relationship to the known values of the output. Thus, it uses the produced function to predict new outputs by assigning new values to the input's independent variable [8].

The classification separates the input information into different categories. Each category is defined by a tag, which is learned by the algorithm. By establishing a relationship between labels and the characteristics of the labeled data, it learns to discriminate future information to make a classification, based on its previous training.

An algorithm that performs clustering generates groups from the input information. The groups are made without the need for training labels. It studies the information and, based on their similarities, groups the data. This ML task is handy to discover categories in a dataset in an unsupervised way [9].

The analyzed information in ML and science, commonly allows humans to predict future events and understand their own world [7]. In general, computer science dedicates its efforts to the search and improvement of algorithms [12] that help humanity to solve problems in the best way. However, in science, it is not all about improving methods. Working with data is vital for science and, therefore, its collection and treatment is one of the main interests in the scientific community. The quality of the data processed in a problem makes sense to the achieved results [12]. Even though collecting the data and preparing it in a dataset is hard, it is a task that the ML cannot skip. The quality of the data is as important as the performance of the algorithm developed.

The information that ML uses is generally structured information. The items that conform to the objects of study and their properties are related utilizing matrices. The preparation of a dataset suitable for the study to be carried out will depend on the user's experience to interpret the data to be able to discriminate those that are useful and eliminate the useless ones.

3 Image classification

ML algorithms that perform item categorization are called classifiers. These algorithms establish a relationship between the elements to be classified $x_i = \{x_1, x_2, \dots, x_m\}$, and the classes or categories $cr_i = \{cr_1, cr_2, \dots, cr_l\}$, where m is the elements to classify, and l y the number of categories. There are different kinds of classification models. Some classifiers yield continuous outputs [4]. This chapter focus just on discrete class labels classifiers. Discrete labels have their limits well-established and do not permit overlapping between two classes. On the other hand, there are various ways to assign labels in a discrete classification. The next lines describe the different type of classifiers, namely [13]:

- Binary: The algorithm has a total of two labels to assign. The classifier labels each input element with one, and only one tag, from the two available.
- Multiple-class: The classifier has three or more labels. The model groups the input elements on one from all the available classes, and assigns the corresponding label. Each element may not belongs to more than one class.
- Multi-labelled: Three or more classes are available to set, but the algorithm is allowed to assign multiple labels to the one element if needed.
- Hierarchical: The input elements are classified by assigning one and only one tag out of the many available. The organization used to categorize the input elements is hierarchically, so there may be classes and subclasses in the classification.

4 Classification metrics

Many parameters influence the performance of an Artificial Neural Network (ANN). The variation in the values of these parameters modifies the performance of the algorithm's design. Due to many parameters and values, the combinations that make the configuration of an algorithm are almost infinite. Knowing the performance of an ANN configuration, respect to another is essential to obtain a network with ideal performance, depending on the classification requirements. Performance metrics are the mathematical tools that help to make an objective comparison between algorithms, according to the results obtained from each ANN. Many tools allow the performance evaluation of classifier algorithms. Sometimes some of these metrics are developed without a clear theoretical justification [5]. This chapter shows some of the most critical metrics that are applied to evaluate the performance of classifiers. These metrics have been extensively studied and analyzed [5] [4] [13], so they present a firm theoretical basis for the study and comparison of classifiers.

4.1 Binary classifier metrics

Evaluating the performance of a classifier is more than just knowing how many input elements were correctly labeled. To understand what each metric evaluates, first must identify the source of its parameters. A confusion matrix is an array of $l \times l$, which compares the classifier's prediction and the real classification of the input data. In this chapter, the original classification labels will be located in the columns and the algorithm's prediction in the rows. Figure 2, shows the confusion matrix of a binary classifier. The following variables are present: The set of real labels $cr_i = \{P, N\}$ and the set of predicted labels $cp_i = \{P, N\}$, where P is the label "Positive" and N represents the "Negative" label. T and F mean *True* and *False* respectively. The meaning of the combinations are described in the next lines, and the Equations 1 to 4 shows their mathematical descriptions.

TP (True Positives) represents the number of elements that belong to the class P and were classified correctly.

TN (True Negatives) is the number of elements that do not belong to the P class and that were correctly classified.

FP (False Positives) denotes the number of elements that refer to class N but were labeled as class P .

FN (False Negatives) expresses the number of elements that pertain to the class P but were labeled as N .

$$TP = \sum_{i=1}^l tp_i \quad \text{where} \quad tp_i = \begin{cases} 0 & \text{if } cr_i \neq cp_i \\ 1 & \text{if } cr_i = cp_i \end{cases} \quad (1)$$

$$TN = \sum_{i=1}^l tn_i \quad \text{where} \quad tn_i = \begin{cases} 0 & \text{if } cr_i = cp_i \\ 1 & \text{if } cr_i \neq cp_i \end{cases} \quad (2)$$

$$FP = \sum_{i=1}^l fp_i \quad \text{where} \quad fp_i = \begin{cases} 1 & \text{if } cr_i = N \otimes cp_i = P \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

$$FN = \sum_{i=1}^l fn_i \quad \text{where} \quad fn_i = \begin{cases} 1 & \text{if } cr_i = P \otimes cp_i = N \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

Cp is the number of elements that really belong to the class P ($TP + FN$) and Cn is the sum of all the elements that really belong to the class N ($FP + TN$).

	$cr_1(P)$	$cr_2(N)$
$cp_1(P)$	TP	FP
$cp_2(N)$	FN	TN
	Cp	Cn

Fig. 2. Confusion matrix for a binary classification.

Accuracy

The accuracy parameter (Equation 5) is the most used metric in classifications. Represents the overall effectiveness of the algorithm in correctly categorizing the input elements. Establishes what percentage of the total elements were properly classified.

$$Acc = \frac{TP + TN}{m} \quad (5)$$

Precision

The Equation 6 calculates the precision of a classifier. This metric gives the percentage from P predictions made by the classifier, are correct.

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

Recall

Recall is a metric that focuses on observing the percentage of the P class elements are correctly labeled by the classifier (Equation 7).

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

F1-Score

Equation 8 shows the mathematical expression to calculate this evaluation metric. *F1score* gives a weighted average between *Precision* and *Recall*. *F1score* comes from F-score [13], but with $\beta = 1$ to give the same importance *Recall* and *Precision*. Except for cases where one of these two variables is really more important, F1-score is most widely used.

$$F1score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (8)$$

4.2 Multi-class classifier metrics

Like in binary classifiers evaluation, it is possible to use Accuracy, Precision, Recall, and F1-score metrics to estimate multi-class classifier's performance. However, having more than two classes enforces to pay more attention to the metrics. Some considerations for the main variants of the most used metrics are discussed ahead. Before that, it is important to see the changes referring to the confusion matrix produced in a multi-class classification to evaluate its performance. Figure 3 shows the composition of a confusion matrix for multi-class classification. The number of classes restricts the use of metrics as defined in Equations 5, 6, 7 and 8, because they are based on two assignation labels, one per class. As multi-class classification has more than two categories, it exceeds the number of labels P and N . When the classifier determines the class membership for an element, the label P is supposed for this class in the confusion matrix, and all other classes are considered N . Similarly, each element that conforms the input dataset will obtain a prediction P just for one class, being N label for the rest of classes.

Figure 3(a) shows the case for a cr_1 class element. In the columns, the label P stands only for cr_1 , and the rest of the classes holds N label. In the predictions axis, just the correct row gets P label, and the rest of them get the label N . As the binary confusion matrix, the combinations TP, TN, FP and FN hold the same arrangement, but different possibilities of prediction. The top-left cell is the only possible place to have the correct prediction. FP predictions increase the number of cells, from one to $l - 1$, and stand in the same row that TP cell. FN cells are the same quantity than TP but are localized vertically to TP to complete the cr_1 column. Finally, the TN cells are localized in the combination points for all the classes, except where cr_1 column or cp_1 row are

present. The number of TN cells are now $(l-1) \times (l-1)$. Figure 3(b) shows the case for labels belonging to class cr_3 . The combination's arrangement is different, but their new distribution corresponds to the same considerations as in Figures 2 and 3(a).

	$cr_1(P)$	$cr_2(N)$	$cr_3(N) \cdots cr_l(N)$			$cr_1(N)$	$cr_2(N)$	$cr_3(P) \cdots cr_l(N)$	
$cp_1(P)$	TP	FP	FP	FP	$cp_1(N)$	TN	TN	FN	TN
$cp_2(N)$	FN	TN	TN	TN	$cp_2(N)$	TN	TN	FN	TN
$cp_3(N)$	FN	TN	TN	TN	$cp_3(P)$	FP	FP	TP	FP
$\vdots$					$\vdots$				
$cp_l(N)$	FN	TN	TN	TN	$cp_l(N)$	TN	TN	FN	TN
	Cp	Cn_1	$Cn_2 \cdots Cn_{(l-1)}$			Cn_1	Cn_2	$Cp \cdots Cn_{(l-1)}$	
	(a) Parameters for $cr_1 = P$					(b) Parameters for $cr_3 = P$			

Fig. 3. Confusion matrix for a multi-class classification evaluation. The variables TP , FN , FP and, TN change depending the element's real class.

Average accuracy

Average Accuracy refers to the average per class effectiveness presented by the classifier. It is usually used when the dataset presents a high unbalance in the elements that belong to each class. When the multi-class dataset samples are balanced, it is common to apply the standard Accuracy.

$$Acc_{avg} = \frac{1}{l} \sum_{i=1}^l \frac{TP_i + TN_i}{m} \quad (9)$$

Macro-average Metrics

Macro-metrics or macro-average metrics calculate each class's metric and then take the arithmetic mean to get the global value. All the metrics follow the same calculus showed in Section 4.1 divided per number of classes, assigning the same weight to each class. Equations 10, 11 and 12 display the mathematical expressions for the macro-average metrics.

$$Precision_M = \frac{1}{l} \sum_{i=1}^l \frac{TP_i}{TP_i + FP_i} \quad (10)$$

$$Recall_M = \frac{1}{l} \sum_{i=1}^l \frac{TP_i}{TP_i + FN_i} \quad (11)$$

$$F1score_M = \frac{2 \cdot Precision_M \cdot Recall_M}{Precision_M + Recall_M} \quad (12)$$

Micro-average Metrics

When using Micro-average-metrics, classes with more elements produce a higher effect on the result than fewer element classes. This effect makes the Micro-average metrics better to evaluate the performance in unbalanced class classification. As some metrics could bring the same value than accuracy for single-label multi-class classification, these metrics are preferable for multi-label classification. Micro-average metrics are displayed as follow: Precision in Equation 13, Recall in Equation 14 and, F1-Score in Equation 15.

$$Precision_\mu = \frac{\sum_{i=1}^l TP_i}{\sum_{i=1}^l TP_i + FP_i} \quad (13)$$

$$Recall_\mu = \frac{\sum_{i=1}^l TP_i}{\sum_{i=1}^l TP_i + FN_i} \quad (14)$$

$$F1score_\mu = \frac{2 \cdot Precision_\mu \cdot Recall_\mu}{Precision_\mu + Recall_\mu} \quad (15)$$

Weighted-Metrics

The weighted metrics take into account the class imbalance. Similar to Micro-average metrics, weighted metrics are useful when the number of elements per class is different. Through Cp_i , it penalizes a class with fewer elements, and benefit a class with more elements. Equations 16, 17 and, 18 shows the mathematical expressions to calculates de weighted metrics. Note that the quotient value is m and not l , as in the macro-average metric.

$$Precision_W = \frac{1}{m} \sum_{i=1}^l \frac{TP_i \cdot Cp_i}{TP_i + FP_i} \quad (16)$$

$$Recall_W = \frac{1}{m} \sum_{i=1}^l \left[\frac{TP_i \cdot Cp_i}{TP_i + FN_i} \right] \quad (17)$$

$$F1score_W = \frac{2 \cdot Precision_W \cdot Recall_W}{Precision_W + Recall_W} \quad (18)$$

5 Neural Nets

5.1 Nodes

An artificial neural network is the implementation of a mathematical algorithm that mimics a brain's functioning in the sense of learning and information processing based on prior learning.

The brain performs these tasks through the interconnection of neurons, through which electric currents circulate, which will be interpreted as ideas. These ideas can be executed at the moment, or they can be stored in some sector of the brain reserved for that specific task. Similarly, a neural network uses an association of permanently interconnected nodes. Information circulates through these nodes in the form of numerical data. The processing of this information means to modify the input data through weights assigned to each connection between nodes.

The animal brain is capable of learning from new information and performing subsequent actions based on previous learning. An artificial neural network initially performs the training work, through which it will establish weight values that determine how the input information is processed. The value of each weight determines the importance that a data represents for a particular node. After the ANN has been trained and its weights established, the network is said to have learned something. The use of these weights, to subsequently process new input information, is the purpose of any neural network.

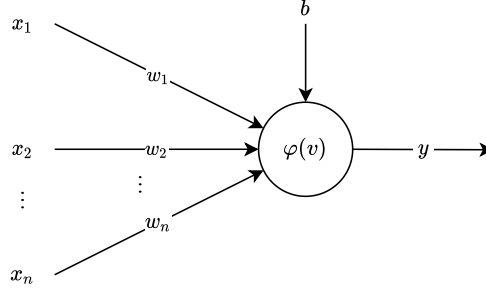


Fig. 4. Graphical representation of a node as a basic unit of a ANN. x represents the data entries, w the weights values to transport the data and b the bias of the node.

Figure 4 shows the essential unit of all ANN. The node is represented by a circle, while arrows represent the flow of information. x represents the input data, w_{ij} represents the weight values between x_j data and the i node, and y_i represents the i node's output. b represents the bias of the node. The node then processes all the input information according to the Equation 19.

$$v_i = \sum_{j=1}^n w_{ij}x_j + b \quad (19)$$

Equation 20 describes the node's output data, where φ represents the node's activation function, and it is the mathematical tool to determine the performance of the node.

$$y_i = \varphi(v_i) \quad (20)$$

As can be seen, there are two essential tools for treating the data at each node. The activation function defines how to process the input information while the weight's values favor or penalizes the data transference between nodes.

5.2 Tensors and data batches

In general, the information that enters an ANN is called a tensor. A tensor is an array that contains the input information of the network, organized in axes, one axis per dimension. Generally, tensors contain numerical data, and when the tensor contains only one scalar data, it is considered a 0D-Tensor. The nature of the input data determines the dimensions that the tensor requires to organize all the information. A data vector is the equivalent of a 1D-Tensor. A matrix is a 2D-Tensor, and an array of matrices would correspond to a 3D-Tensor.

The input information of an ANN receives the name as a whole dataset. Nowadays, the information that can be collected for processing is considered significant. However, working with big datasets is difficult and more laborious [12]. Big datasets are often divided into smaller units of information called batches. The information batches that neural network processes are the real input data of a neural network.

Having a neural network, where the input information is a scalar, is wholly absurd. ANNs are tools constructed to facilitate the work of processing large numbers of data. That is the reason an ANN receives a 1D-Tensor as the smallest input shape of data. For example, this vector can store the information about the weight in kilograms of 100 people so that it would be represented as a one-dimensional arrangement of 100 elements.

A 2D-Tensor can contain more information: weight, glucose level, and blood pressure of a group of people under medical study. If the group it is composed by up of 100 individuals, the 2D-Tensor would have the form (100, 3), where the first element represents the size of the data batch, and the second element corresponds to the number of data contained in the information vector (1D-Tensor) of each of the 100 people.

In the previous example, we can add the measurement of the variables stored out every 24 hours. In that case, the data would be collected in a 3D-Tensor. Suppose, the medical personnel analyses the collected data every month. The data batch takes the form (100, 30, 3), where the second

element is a new component added to the previous 2D-Tensor. This last element represents that the information comes in batches of 30 samples, one for each month's day. Each sample contains the information of the three variables measured in each one of the persons under study.

To exemplify a 3D Tensor as input on a data batch in an ANN, we resort to an image dataset as the input data. In this case, we divide the original dataset into smaller image groups. Each image is itself a three-dimensional Tensor: height, width, and color depth. Depending on the type of image, it may contain more or fewer layers of color. A grayscale image has only one layer. A color image usually has three layers, one for red values, a second layer for green values, and the third for blue values. For processing a dataset of 300 color images of 100×150 pixels, we can divide it into batches of 30 images. From that division, we obtain a 4D-Tensor in the form $(30, 100, 150, 3)$, which means groups of 30 images, of a dimensional space of 100×150 pixels, and three layers of colors.

Finally, a 5-dimensional data batch represents the input data of a video. A 30-second video, with 24 samples per second and a 144×256 spatial dimension, can be represented by a 5D-Tensor of the form $(24, 720, 144, 256, 3)$. In this case, the second element corresponds to the 720 frames ($24 \text{ samples} \times 30 \text{ seconds}$) of the video. Thus, Table 1 shows the data types contained in tensors of different dimensions for an ANN.

Table 1. Tensors by dimensions and examples of its contents.

Name	Dimensions	Example	Shape and content
1D Tensor	1	Vector of scalars	(samples)
2D Tensor	2	Matrix	(samples, features)
3D Tensor	3	Timeseries data	(samples, timesteps, features)
4D Tensor	4	Images	(samples, height, width, channels)
5D Tensor	5	Video	(samples, frames, height, width, channels)

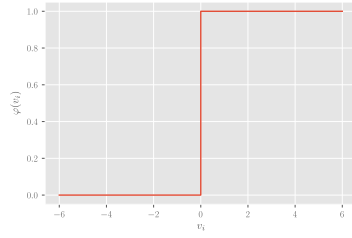
5.3 Activation function

The activation function is a mathematical equation that modifies the value of v for each node. Its purpose is to normalize y in a cipher between two determined thresholds. Due to the number of activation functions that a neural network contains, and considering almost all of them are calculated in each training, the mathematical representation of the activation function must be computationally efficient. In this way, it is common to use discrete functions or functions with simple derivatives. In the next lines are listed the most popular activation functions:

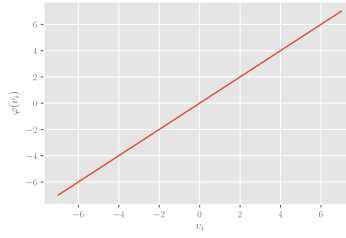
Binary step function

The simplicity of this activation function lies in having a threshold value t . The node output then corresponds to one of two possible values, depending on whether v value is higher or lesser than t (Equation 21). The graphic function is shown in Figure 5a, where $t = 0$. Due to its simplicity, the primary use of this function is to categorize data in binary classification.

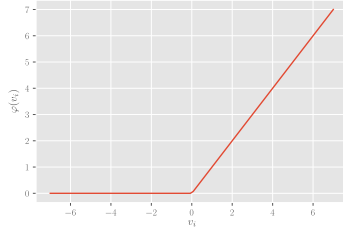
$$\varphi(v_i) = \begin{cases} 0, & \text{for } v_i < t \\ 1, & \text{for } v_i \geq t \end{cases} \quad (21)$$



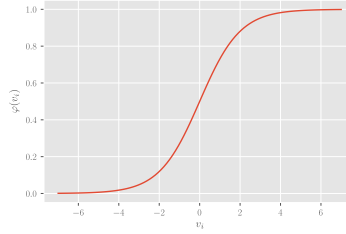
(a) Binary step function



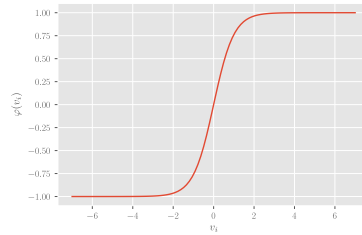
(b) Linear function



(c) ReLU function



(d) Sigmoid function



(e) Hyperbolic tangent function

Fig. 5. Graphics of some of the most popular activation functions.

Linear function

This function delivers a signal proportional to v at the node output as the Equation 22 shows. Unlike the Binary step function, the linear function does not limit the node's output to only two possible values. However, it has the drawback that its derivative is a constant, so there is no relationship between the nodes that help in more significant measures to improve network predictions. Similarly, using only the Linear Function in all the nodes of an ANN, ends up summarizing a multilayer neural network's operation as a single layer neural network [10]. The Figure 5b shows the behavior of this function.

$$\varphi(v_i) = v_i \quad (22)$$

Rectified Linear Unit (ReLU)

The ReLU function is expressed mathematically in Equation 23. Due to its performance, the ReLU function is one of the most used activation functions in artificial neural networks. Its best achievement concerning the Linear Function lies in the elimination of negative values as Figure 5c shows. This feature gives the ANN the benefits of a linear function for all positive values, but by eliminating values less than zero, it breaks the total linearity of the function. This attribute brings ReLU the opportunity to use it in all layers of the network.

$$\varphi(v_i) = \begin{cases} 0, & \text{for } v_i < 0 \\ v_i, & \text{for } v_i \geq 0 \end{cases} \quad (23)$$

Sigmoid or Logistic

Because its output is bounded in the range $(0, 1)$, it is a useful function in networks that deliver probability values at their output. It is a function without sudden jumps that allows the net to obtain good results for classification tasks, even when the computational cost of its use is higher compared to ReLU. The Equation 24 shows its mathematical definition and Figure 5d shows its graphical response.

$$\varphi(v_i) = \frac{1}{1 + e^{-v_i}} \quad (24)$$

Hyperbolic Tangent (TanH)

TanH function has almost the same characteristics of Sigmoid Function. The most notable difference is showed in Figure 5e, where is visible that the TanH function presents a possibility of output values in the range $(-1, 1)$. Its mathematical definition can be seen in the Equation 25.

$$\varphi(v_i) = \frac{e^{v_i} - e^{-v_i}}{e^{v_i} + e^{-v_i}} \quad (25)$$

Softmax

The Softmax function generates an output vector as the probability distribution of the input vector. The output elements are in the range (0, 1), and the sum of the total elements of the generated vector will always be equal to unity. The Equation 26 shows the Softmax function. Due to its characteristics, it is a function widely used for classification tasks, working in the output layer of the ANN.

$$\varphi(v_i) = \frac{e^{v_i}}{\sum_{i=1}^n e^{v_i}} \quad (26)$$

5.4 Layers

As seen in previous pages, a node is the basic processing data unit of machine learning based on neural networks. It is essential to establish a way through the nodes for information flows, from the beginning to the end of the network. In reality, the information flows through a net instead of a single way. The nodes that conform to a neural network are placed in layers, which could be defined as a one-dimensional array of nodes, where no node has communication with another node on the same layer.

The layer that receives through itself the information to be processed in the neural network is called the input layer. By acting as a gateway, the input layer does not perform any mathematical calculations. After the input layer comes the hidden layers, which have no communication to the outside of the network. Its function is to process the data received from the input layer, through the previously seen nodes and activation functions. All of the nodes in the input layer have a direct connection to all of the nodes in the first hidden layer. In the same way, it happens between the nodes of the first hidden layer and the second, and so on until we reach the last layer of the neural network. This last layer is called the output layer. The output layer is in charge of carrying out the last treatment of the information and delivers to the user the result of the data treatment, as the network has been designed.

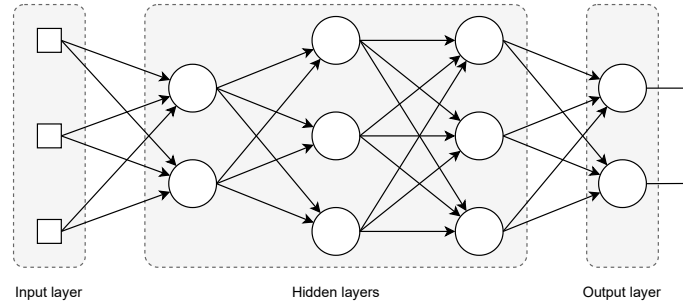


Fig. 6. A basic artificial neural network is made up of three types of layers based on their position relative to the exterior of the network: the input layer, hidden layers, and the output layer.

Figure 6 shows the general structure of a neural network: the input layer, hidden layers, and the output layer. Figure 6 also shows the nodes interconnection between layers, where each node in a layer receives information from all nodes in the previous layer, except the input layer. It is also observed how each node sends its output information to all nodes in the next layer, except the output layer. It is from this interconnection characteristic that these node layers are called fully connected layers.

5.5 Supervised learning

Supervised learning refers to the training of an ANN using a database with known information. The network works with an association of data and labels so that it knows what result to expect at the output layer. The training of the network is then carried out in a supervised manner, which means, it is verified if the output data corresponds to its labels; otherwise, adjustments are made in the weights of the network to obtain the closest result to the desired optimum. The general steps for supervised learning are outlined below, and Figure 7 shows the general flowchart for supervised learning.

1. Start the training with the input data vector, assigning random value weights. The output vector is obtained at the end of the network.
2. The output vector is compared to the ideal output vector. The difference between real output and the desired output sets the output error.
3. A readjustment is made in the values of the weights that interconnect the nodes.
4. The process is repeated until the training stop criterion is reached, which is normally a certain number of iterations called epochs.

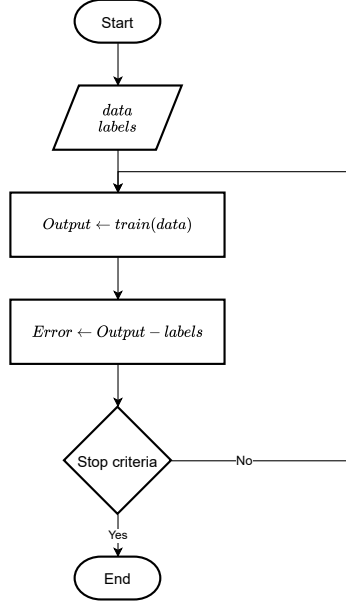


Fig. 7. Flow diagram of the general supervised learning process.

5.6 Learning rate

At this point, it is necessary to consider the existence of the learning rate, which determines the dimension of the change to be made in the weights of the nodes when supervised learning is carried out. Applying the supervised learning algorithm in a single node, the calculation of step 2 can be seen in the Equation 27, where e_i corresponds to the error, d_i corresponds to the ideal output and y_i is the output of the node after applying the Equation 20. The execution of step 3 of supervised learning performed in a generic way, can be seen in the Equation 28. Here, we can see how updating each of the weights is done based on the dimension of the error. In a similar way, the learning rate influences the update. The difference between the learning rate and the error is that the learning rate remains constant throughout the training process with a value chosen by the user in the range $(0, 1)$, unlike the error, the magnitude of which is sought to decrease on each iteration until ideally zero.

$$e_i = d_i - y_i \quad (27)$$

$$w_{ij} = w_{ij} + \alpha e_i x_j \quad (28)$$

6 Convolutional Neural Networks

Convolutional Neural Networks are artificial neural networks that, in addition to fully connected layers, includes one or more convolutional layers. The purpose of the convolutional layers is to extract the main characteristics of the input tensor's data. This process is close to the location of the features into an image by the animal eye. CNN also contains typically subsampling layers that help to reduce the dimensions of the delivered tensor to the last inner layer. At the end of the hidden layers section, a fully connected layer is placed, which flattens the data to prepare it in a manner that the output layer can classify the information. Figure 8 shows the main stages of a CNN, as well as a hypothetical and possible arrangement.

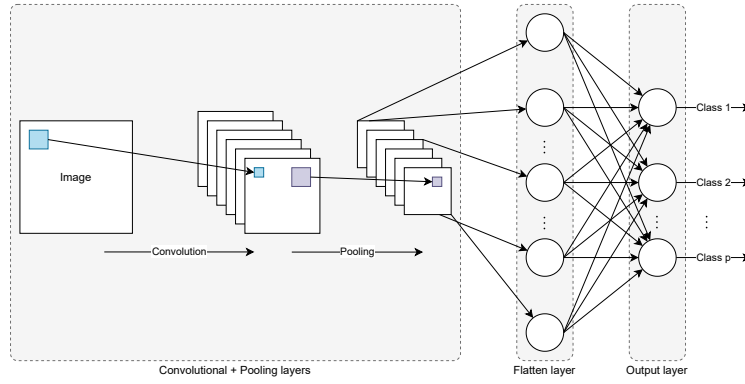


Fig. 8. CNN's adds the convolutional and pooling layer to perform data feature extraction before the classification.

6.1 Convolutional layer

CNN uses the convolutional layers to extract the data features from the input data. The layer's name came from the task that it performs. This layer convolves a kernel and the input data. The convolution kernel executes a series of sweeps in the whole input tensor. In each sweep, the tensor and the kernel convolves with a different filter.

$$conv = \sum_{i=1}^r \sum_{j=1}^c K_{ij} I_{ij} \quad (29)$$

Assume a single layer image, as illustrated in Figure 9. The kernel size is (3×3) pixels. The kernel has a filter, and it poses an area of equal dimensions over the image. The convolution starts applying the Equation 29 in this position. I represents the kernel's elements, and K describes the image area covered by the kernel. r and c represents the number of rows and columns

of both matrices. The result of the Equation 29 is assigned at the first position of the feature map. Thus, the kernel swaps one position, and applies the Equation 29 again. The convolution finishes when the kernel passes over all the tensor positions.

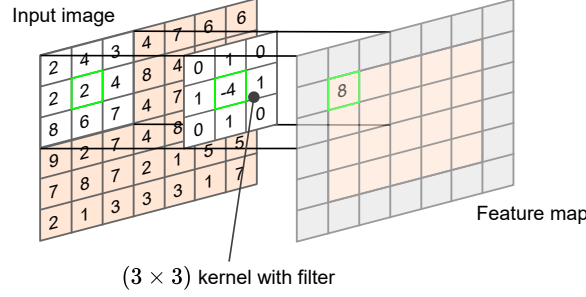


Fig. 9. The kernel and image convolution generates a new output tensor, which contains the image’s features. This output tensor is called feature map. The feature map suffers a spatial shrink by lost the edge pixels.

When carrying out the convolutions, there remains a series of pixels with no assigned value in the edge of the feature map (Figure 9). These no value positions suppose a dimensional reduction of the output tensor concerning the input tensor.

While the convolutions are performed, it produces the equivalent of a new image from each filter. If it convolves 16 filters within an image, it produces a total of 16 images. Each image will present $r - 2$ rows and $c - 2$ columns and highlight the original image’s specific characteristics. Separate and highlight the image’s characteristics brings the opportunity to discriminate information and be able to distinguish between the different objects contained in the graph.

6.2 Pooling layer

Working high volume data increases the computational cost. Work with small, still significant data, improves to explore and analyze the information faster [12]. The pooling layer produces a spatial reduction of the tensor generated by the convolution layer, trying to preserve the information. It divides the tensor into windows. The most common window size is 2×2 pixels, reducing the image area four times. The most common application extracts the maximum or average value from each window [6]. Sometimes the minimum value is used instead. Figure 10 shows the spatial reduction of an 8×6 pixels image to a 4×3 pixel image by sub-sampling maximum values with 2×2 pixel windows. The spatial reduction of each layer of the feature map is an extract of the high-level characteristics of the image [6]. Each time the CNN performs

the subsampling operation, it obtains a less position-dependent feature map, while reducing the amount of memory required for image processing in the subsequent layers [11].

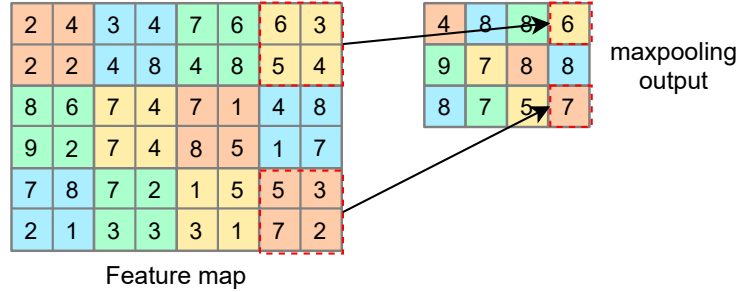


Fig. 10. The pooling layer reduces the the feature map spatial dimension. It extracts representative information to conform a new high level data feature map.

6.3 Fully connected layers

These layers are the exit stage of the neural network and are where the classification task is carried out. In this stage, CNN performs two central processes. The first one is the elimination of the spatial information from the data through the change of tensor's dimension, where the output tensor of the last pooling layer will be transformed into a 1D-Tensor to be entered in a fully connected layer. The second is the probability distribution assignment for the classes contained in the network's input tensor. The assignment takes place in the neural network output layer through the use of a softmax activation function.

7 Training CNN

Today, implementing CNN is easy. The adaptation of programming languages and the development of specific libraries for this task facilitates its construction. Python is one of the widely used languages for neural network programming. The development of modules and libraries written in Python, such as Keras and Scikit Learn, facilitates using specialized libraries such as Tensor Flow. With these libraries becomes comfortable and practical, the use of a general programming language, like Python, to perform specialized machine learning tasks

Since the central theme of the chapter is the development of a CNN structure for classifying images, it does not cover the basic concepts of programming with Python language, nor does it deepen the understanding of Tensor Flow

libraries [3], Keras [14] or Scikit [2], are only used as tools to achieve the purpose of the chapter. To deepen Python's study or any of the libraries, refer to the respective user manual.

The classification tests performed in this chapter use the images from the MNIST dataset [1]. MNIST contains images of handwritten digits of 28×28 pixels. The dataset contains ten classes, one for each digit from 0 to 9 (Figure 11). The whole set consists of 70,000 images divided into two subsets, the first of 60,000 for training and the second of 10,000 elements for validation. For this work, the validation set was divided. A total of 14 images were extracted from each class for further testing of the algorithm. In this way, the confusion matrix was obtained, and the metrics were calculated to compare the trained algorithms.

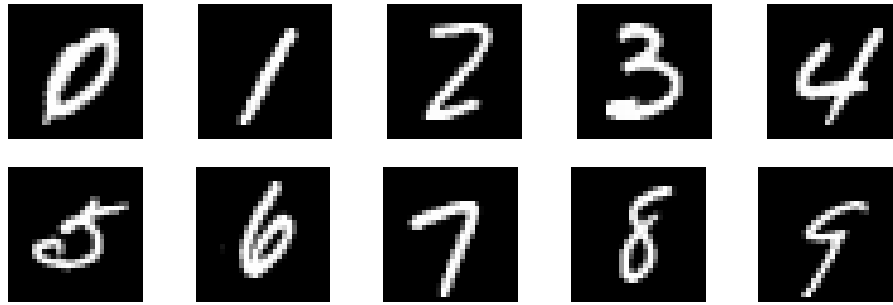


Fig. 11. Images from MNIST database. Each image contains one 0 to 9 digit in it.

There are many types of neural networks, specialized according to the task it must perform. In this chapter, convolutional neural networks for a multi-class classification are discussed. The objective is to classify the images in a dataset and observe the trained algorithms' performance.

The general structure of a CNN (Section 6) is easy to understand and implement. However, depending on the characteristics of the images to be classified, it is necessary to make adjustments that require the designer's experience and expertise. On the one hand, designers know the task performed by each specified parameter in the neural network, but on the other hand, it is difficult to understand the effect generated by the interaction of the parameters with the different possible values. Understanding this interaction requires much experience. Even an expert designer needs to perform tests to achieve a proper tuning of the algorithm to classify the images correctly.

Trying to show the effect when a parameter changes its value, the basic CNN structure (Table 2) was tried with nine different parameter values combinations. Each combination was trained ten times, and the trained CNN was stored, as its classification results. From the stored data, the confusion ma-

trix was plotted, and then, the evaluation metrics calculated. The Section 8 presents the arithmetic mean of the all the times an algorithm was trained.

Table 2 shows the layers of the test algorithm. A total of 9 layers are sufficient to obtain good classification results with the MNIST dataset, due to the high contrast of the images and the low spatial resolution. When images have a higher spatial resolution, adding more layers helps to gradually decrease the data dimensions so as not to have significant information losses. With images that present many details such as faces or complex objects, it is necessary to add more convolutional layers to extract more characteristics that facilitate a better image classification. Sometimes, it is necessary to change CNN's structure due to the complexity that the images present.

Programming a CNN in Python begins by loading the modules necessary to work (Listing 1) such as Tensor Flow, Sklearn, and Keras. Besides the special modules for machine learning, the algorithm requires other basic ones for working with graphics, numerical arrays, or manipulate files.

Listing 1. Python code to import the modules needed to work.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sys
import os
from tensorflow import keras
import tensorflow as tf
from tensorflow.python.keras.preprocessing.image
    import ImageDataGenerator
from tensorflow.python.keras.applications.mobilenet
    import preprocess_input
from tensorflow.python.keras import optimizers
from tensorflow.python.keras.models import Sequential
from tensorflow.python.keras.layers import Dropout,
    Flatten, Dense, Activation
from tensorflow.python.keras.layers import Convolution2D,
    MaxPooling2D
from tensorflow.python.keras import backend as K
from keras.preprocessing import image
```

To image classification, the algorithm must access the images dataset and extract the elements. The dataset can be hosted on a server or locally on the computer. In this case, the dataset is hosted locally, so we must specify the directory address for training, validating, and testing images (Listing 2, lines 1 to 3). In line 4, the algorithm gets the class labels from the folder's names. The train, validate and test folders, has inside one folder per class, and the name of each folder will be the corresponding label. Sometimes, images are not uniform and need some preprocessing. `ImageDataGenerator()`

and `flow_from_directory()` perform a preprocessing of images. These tasks include the normalization of pixel values from 0-255 to 0-1; uniform, if not, the spatial dimension of the images; and group the images in batches of a determined size for the learning process. Sometimes, this preprocessing includes rotating the images to generate more elements to train the algorithm properly. In Listings 2, the preprocessing task ensures all the images are 28×28 pixels, and the batch size is 32 images each for training and validating image subsets.

Listing 2. Code to call and preprocessing the images to classify.

```

1 training_folder = './dataset_folder/train '
  validation_folder = './dataset_folder/validate '
3 test_folder = './dataset_folder/test '
  training_folder_names = os.listdir(training_folder)
5
  training_datagen = ImageDataGenerator(rescale=1./255)
7 validation_datagen = ImageDataGenerator(rescale=1./255)

9 train_images = training_datagen.flow_from_directory(
    training_folder ,
11     target_size=(28, 28),
    batch_size=32,
13     class_mode='categorical ')
  val_images = validation_datagen.flow_from_directory(
15     validation_folder ,
    target_size=(28, 28),
17     batch_size=32,
    class_mode='categorical ')
19
  training_steps = train_images.n//train_images.batch_size
21 validation_steps = val_images.n//val_images.batch_size

```

Building the CNN structure with Keras means adding the neural net layers and assigning each one the desired characteristics. Listing 3 shows the algorithm's definition used to classify the MNIST dataset images. Line 1 starts the algorithm building. `Convolution2D()` and `MaxPooling2D()` sets the respective sort of layer. The designer sets the parameters for each layer independently.

Line 3 sets a 32 convolutional kernels of 3×3 size each. `padding='valid'` means that produced feature map will suffer a dimensional reduction (Figure 9). When `padding='same'` Keras will convolute the images and the kernels producing a feature map with the same spatial dimensions that the input tensor. Line 3 shows the specification of the images' dimensions: 28×28 pixels and three color layers. The activation function is specified too. By default, if not stipulated, Keras uses the ReLu activation function for convolutional

layers. Line 4 shows the max-pooling layer definition. In this case, specify the 2×2 window size.

Lines 6 to 10 define the next layers, increasing the convolutional kernels' number to 64 and 128. Increasing the number of kernels allow CNN to extract more information from the tensors. More information gives the algorithm the possibility to learn better and classify better.

Listing 3. Definition of the CNN's structure.

```

1 cnn = Sequential()
  cnn.add(Convolution2D(32, (3,3), padding='valid ',
3      input_shape=(28, 28, 3), activation='relu '))
  cnn.add(MaxPooling2D(pool_size=(2,2)))
5
  cnn.add(Convolution2D(64, (3,3), padding='valid '))
7 cnn.add(MaxPooling2D(pool_size=(2,2)))

9 cnn.add(Convolution2D(128, (3,3), padding='valid '))
  cnn.add(MaxPooling2D(pool_size=(2,2)))
11
  cnn.add(Flatten())
13 cnn.add(Dropout(0.5))
  cnn.add(Dense(256, activation='relu '))
15 cnn.add(Dense(10, activation='softmax '))

17 cnn.summary()

19 cnn.compile(loss='categorical_crossentropy ',
              optimizer=optimizers.Adam(lr=0.0003),
21              metrics=['accuracy '])

23 H = cnn.fit(
      training_images ,
25      steps_per_epoch=training_steps ,
      epochs=4,
27      validation_data=validation_images ,
      validation_steps=validation_steps)

```

Lines 10 to 15 add the last algorithm's layers. Set the fully connected layer to 256 nodes, and the softmax layer to 10 nodes, one per class. The dropout turns off some layer nodes in a random form to avoid the neural net overfitting. Line 13 adds a dropout of 0.5 to the fully connected layer, which means it will turn off 50% of its nodes. Line 17 displays a summary of the designed CNN. `compile()` configures the constructed model for the training process. It sets the objective function, the optimizer, learning rate, and the metrics to evaluate the CNN training. `fit()` trains the compiled model. Receive as

Table 2. Layers composition of the basic classification algorithm.

Layer	Layer type	Output shape
1	Convolution layer 1	26x26x32
2	Max Pooling 1	13x13x32
3	Convolution layer 2	11x11x64
4	Max Pooling 2	5x5x64
5	Convolution layer 3	3x3x128
6	Max Pooling 3	1x1x128
7	Flatten layer	128
8	Full connected layer	256
9	Softmax layer	10

arguments, the preprocessed images for training and validating, the epochs or iterations, and the steps. The number of steps is the number of samples in that an epoch is divided. It can be understood like the unit of information to process during the training and validation processes.

When the algorithm completes a training epoch, it measures the training accuracy and validation accuracy to evaluate the learning process. The process also calculates training loss and validation loss. Loss is the objective function's evaluation of the training and the validation and represents the sum of the errors made by each example. Listing 4 shows the code implementation to plot the historical changes for loss and accuracy through the training process. Matplotlib, is a python's module used for plotting this kind of graphics.

Listing 4. Code for printing the historical of Loss and Accuracy for the training process.

```

plt.style.use("ggplot")
2 plt.figure()
  plt.plot(np.arange(0, Epc), H.history["loss"],
4           label="train_loss")
  plt.plot(np.arange(0, Epc), H.history["val_loss"],
6           label="val_loss")
  plt.plot(np.arange(0, Epc), H.history["accuracy"],
8           label="train_accuracy")
  plt.plot(np.arange(0, Epc), H.history["val_accuracy"],
10          label="val_accuracy")
  plt.title("Training Loss and Accuracy on Dataset")
12 plt.xlabel("Epochs")
  plt.ylabel("Loss/Accuracy")
14 plt.legend(loc="center right")

```

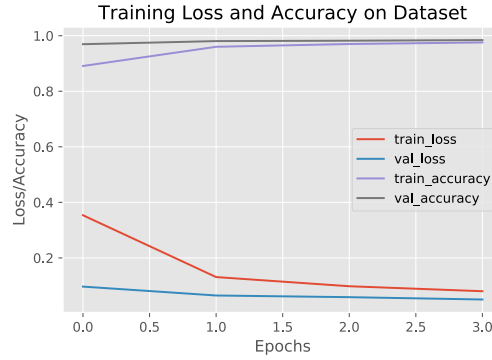


Fig. 12. Training and validation metrics historical plot.

Once the algorithm's training process finishes, it is time to determine its effectiveness, using the test images subset. First, the images are preprocessed (Listing 5), adapting the parameters of `flow_from_directory()` for the classification task instead of a training task. Now, the algorithm has learned the train images subset characteristics, supposing it learned enough to classify images it has never seen before. `batch_size=1` means the algorithm will process image by image. Lines 15 to 22 perform the test images subset classification using the algorithm previously trained. Once the CNN classifies the whole subset, the algorithm stores the results in a .csv file. The collection of .csv files will be processed to perform each training's matrix confusion for each different algorithm. Then, the evaluation metrics are calculated.

Listing 5. Programming code for test images classification in the trained algorithm.

```

test_datagen = ImageDataGenerator(
2     preprocessing_function=preprocess_input)

4 test_generator = test_datagen.flow_from_directory(
    test_folder ,
6     target_size=(28, 28),
    color_mode="rgb",
8     batch_size=1,
    class_mode=None,
10    shuffle=False ,
    seed=42)

12 test_steps=test_generator.n//test_generator.batch_size
14 test_generator.reset()
pred=cnn.predict(test_generator , steps=test_steps ,
16     verbose=1)
predicted_class_indices=np.argmax(pred , axis=1)

```

```

18 labels = (training_images.class_indices)
20 labels = dict((i, j) for j, i in labels.items())
    predictions = [labels[j] for k in
22                  predicted_class_indices]

24 filenames = test_generator.filenames
    results = pd.DataFrame({"Filename": filenames,
26                          "Predictions": predictions})
    results.to_csv("./redes/Predictions.csv", index=False)

```

8 Results

Nine algorithms with different parameter values were trained and compared. Table 3 presents the parameter values of each algorithm. The parameters presented in the table are not the whole but are the parameters that differentiate between algorithms. To observe the change in the classification performance in a better form, we should observe and compare those algorithms which change one parameter value between them. The Table 4 shows the pair of algorithms to compare by parameter in graphical form. So to see the differences when changing the epochs number, compare algorithms B and D. To see the difference between algorithms with different batch sizes to process, compare A and F. For padding and dropout, are two pairs to compare. A versus D, and C versus E for padding. Dropout differences are between A and C, and the second pair is D and E. The last case, the activation function, is not a comparison between two, but between three. Algorithms A, G, and H have de same parameter values, except the activation function.

Table 3. Parameter values used by algorithm.

Variables	Algorithms							
	A	B	C	D	E	F	G	H
Epochs	4	8	4	4	4	4	4	4
Batch Size	32	32	32	32	32	16	32	32
Padding	same	valid	same	valid	valid	same	same	same
Dropout	No	No	0.5	No	0.5	No	No	No
Activation function	Relu	Relu	Relu	Relu	Relu	Relu	tanh	sigmoid

Table 5 shows the result of the metrics taken during or after the training process. Measured variables are Total training time in seconds, Milliseconds per step as measure unit, Training accuracy, and Validation accuracy. Due

Table 4. Groups of algorithms to compare by parameter.

Comparing	Algorithms							
	A	B	C	D	E	F	G	H
Epocs			•		•			
Batch size	•						•	
Padding	*		•	*	•			
Dropout	•		•	*	*			
Activation Func	•						•	•

to the total time is a subjective variable for algorithms with different Epocs, the *ms/step* is presented to compare the time to process one step or work unit during the training. Each variable presents its arithmetic mean and the deviation standard. All the Arithmetic mean columns present its better value in bold and its worst value in underline.

Table 5. Metric results during training process.

Alg.	Train. time		ms/step		Training accuracy		Validation accuracy	
	Mean	Std. D.	Mean	Std. D.	Mean	Std. D.	Mean	Std. D.
A	842	66.204	112.275	8.907	0.993266	0.000262	0.990807	0.00115
B	<u>1616.5</u>	292.606	107.737	19.558	0.997125	0.000182	0.991477	0.000948
C	889.2	127.493	<u>118.6</u>	17.019	0.988196	0.000535	0.991	0.00161
D	518.9	38.988	69.175	5.196	0.986843	0.000465	<u>0.983015</u>	0.002647
E	507.9	4.988	67.675	0.725	<u>0.975293</u>	0.000621	0.983969	0.001391
F	986.6	38.897	65.75	2.605	0.993846	0.000349	0.990614	0.000894
G	779.4	3.693	101.525	7.02	0.993145	0.000502	0.990056	0.001156
H	793.8	35.521	105.85	4.718	0.989789	0.000345	0.988392	0.001466

To compare the performance of two classifiers, we got to use the classification metrics. All the algorithms performed the classification of the same image subset. One hundred forty images, 14 per class compose test images subset. The predictions gathered were compared with the real class for each element in a confusion matrix. The values of *TP*, *FN*, *FP* and, *TN* were used to calculate the Accuracy and the Macro-averaged metrics because it is known the classes are balanced. Once the metrics where calculated, the arithmetic mean and the standard deviation was extracted and listed on Table 6.

9 Conclusions

Comparing the couples and threesome of algorithms, its clear that there is no definitive best combination but depends on the classification requirements

Table 6. Results of classification metrics when comparing trained algorithms.

Algorithm	Average accuracy		Macro precision		Macro recall		Macro F1-Score	
	Avg	Std. Dev.	Avg	Std. Dev.	Avg	Std. Dev.	Average	Std. Dev.
A	0.994285	0.004948	0.971428	0.024743	0.976069	0.019274	0.973734	0.022023
B	0.995571	0.003467	0.977857	0.017335	0.980758	0.013838	0.979302	0.015589
C	0.994142	0.008092	0.970714	0.040462	0.977412	0.02865	0.97401	0.034636
D	0.981857	0.015186	0.909285	0.075932	0.938035	0.038493	0.922953	0.058127
E	<u>0.975285</u>	0.016438	<u>0.876428</u>	0.082192	<u>0.915822</u>	0.046607	<u>0.895222</u>	0.065372
F	0.995285	0.003672	0.976428	0.018364	0.979137	0.015608	0.977779	0.01699
G	0.977571	0.010918	0.887857	0.05459	0.921869	0.031009	0.904346	0.043075
H	0.993142	0.002695	0.965714	0.013477	0.969695	0.011043	0.967698	0.012247

and designer’s goals. If we augment the epochs, training time will augment, sometimes takes weeks or months in it, depending on the complexity of the algorithm. On the other hand, efficiency is clear. The question is whether the result or the application takes away the training time. In the batch size variable, a size of 16 has better results in the classification comparative, but during the training, the batch size of 32 performs better. For both comparatives for padding, it is clear that keep the value in *same* gets the best results in classifications but works slower during training. For dropout, it is different. There is no clear advantage or disadvantage if dropout is active. Being a variable which turns off the nodes depending on the randomness, it is preferable to use it and adjust the percentage, if the algorithm tends to the overfitting. To the last, the activation function comparative shows why is ReLu function the most popular for this purpose.

Any parameter works alone. It is important, first, to know the nature of the images to classify. Knowing the tendencies of some parameters is a good guide to set the first parameter values and then syntonize depending on the gathered results. The most important parameter is not inside the algorithm, but in the designer: the expertise and the theoretical bases to conduct his/her adjustments in the algorithms.

References

- [1] (2013) MNIST handwritten digit database, Yann LeCun, Corinna Cortes and Chris Burges. URL <http://yann.lecun.com/exdb/mnist>, [Online; accessed 28. Jun. 2020]
- [2] (2020) scikit-learn: machine learning in Python — scikit-learn 0.23.2 documentation. URL <https://scikit-learn.org/stable>, [Online; accessed 28. Aug. 2020]
- [3] (2020) TensorFlow. URL <https://www.tensorflow.org>, [Online; accessed 28. Aug. 2020]

- [4] Fawcett T (2006) An introduction to roc analysis. *Pattern Recognition Letters* 27(8):861 – 874, DOI <https://doi.org/10.1016/j.patrec.2005.10.010>, URL <http://www.sciencedirect.com/science/article/pii/S016786550500303X>, rOC Analysis in Pattern Recognition
- [5] Ferri C, Hernández-Orallo J, Modroiu R (2009) An experimental comparison of performance measures for classification. *Pattern Recognition Letters* 30(1):27 – 38, DOI <https://doi.org/10.1016/j.patrec.2008.08.010>, URL <http://www.sciencedirect.com/science/article/pii/S0167865508002687>
- [6] Guo T, Dong J, Li H, Gao Y (2017) Simple convolutional neural network on image classification. In: 2017 IEEE 2nd International Conference on Big Data Analysis (ICBDA), pp 721–724, DOI 10.1109/ICBDA.2017.8078730
- [7] Igual L, Seguí S (2017) *Supervised Learning*, Springer International Publishing, Cham, pp 67–96. DOI 10.1007/978-3-319-50017-1_5, URL https://doi.org/10.1007/978-3-319-50017-1_5
- [8] Igual L, Seguí S (2017) *Supervised Learning*, Springer International Publishing, Cham, pp 67–96. DOI 10.1007/978-3-319-50017-1_5, URL https://doi.org/10.1007/978-3-319-50017-1_5
- [9] Igual L, Seguí S (2017) *Unsupervised Learning*, Springer International Publishing, Cham, pp 115–139. DOI 10.1007/978-3-319-50017-1_7, URL https://doi.org/10.1007/978-3-319-50017-1_7
- [10] Kim P (2017) *Neural Network*, Apress, Berkeley, CA, pp 19–51. DOI 10.1007/978-1-4842-2845-6_2, URL https://doi.org/10.1007/978-1-4842-2845-6_2
- [11] Reinders C, Ackermann H, Yang MY, Rosenhahn B (2019) Chapter 4 - learning convolutional neural networks for object detection with very little training data. In: Yang MY, Rosenhahn B, Murino V (eds) *Multimodal Scene Understanding*, Academic Press, pp 65 – 100, DOI <https://doi.org/10.1016/B978-0-12-817358-9.00010-X>, URL <http://www.sciencedirect.com/science/article/pii/B978012817358900010X>
- [12] Skiena SS (2017) *What is Data Science?*, Springer International Publishing, Cham, pp 1–25. DOI 10.1007/978-3-319-55444-0_1, URL https://doi.org/10.1007/978-3-319-55444-0_1
- [13] Sokolova M, Lapalme G (2009) A systematic analysis of performance measures for classification tasks. *Information Processing & Management* 45(4):427 – 437, DOI <https://doi.org/10.1016/j.ipm.2009.03.002>, URL <http://www.sciencedirect.com/science/article/pii/S0306457309000259>
- [14] Team K (2020) Keras: the Python deep learning API. URL <https://keras.io>, [Online; accessed 28. Aug. 2020]